C O M P L E T E L E A R N I N G M O D U L E

Playwright

with JavaScript

From Zero to Production-Ready Automation Framework

21	3	4h	3
Days	Weeks	Per Day	Projects

What you will master:

Node.js Setup • JavaScript async/await • Locators • Assertions • Waits • Advanced UI • Debugging • Page Object Model • Fixtures • Data-Driven Tests • API Testing • CI/CD with GitHub Actions

---

Prerequisites: Basic computer literacy. Node.js installed (v18+). VS Code recommended. No prior Playwright or automation experience needed.

Table of Contents

SECTION 1 — PREREQUISITES & ENVIRONMENT SETUP

Prerequisites & Environment Setup

Before writing a single line of Playwright code, your development environment must be correctly configured. This section walks you through every step — from installing Node.js to understanding your first project's folder structure.

1.1 Node.js Installation

Playwright requires Node.js version 18 or higher. Node.js is the JavaScript runtime that lets you run JS code outside of a browser — it powers npm (the package manager) and the Playwright test runner.

How to install Node.js

1. Go to https://nodejs.org and download the LTS (Long-Term Support) version.
2. Run the installer. On Windows it adds Node.js to your PATH automatically. On macOS/Linux you may need to use the pkg installer or a version manager.
3. Verify installation by opening a terminal and running:

```
node --version    # should print v18.x.x or higher
npm --version     # should print 9.x.x or higher
```

4. (Recommended) Install nvm — the Node Version Manager — to easily switch between Node versions:

```
# macOS / Linux
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
nvm install 20
nvm use 20

# Windows — use nvm-windows from:
# https://github.com/coreybutler/nvm-windows/releases
```

TIP

Why LTS?

LTS versions receive security fixes for 3 years. Playwright's CI runners use LTS versions by default, so using LTS locally avoids version mismatch issues.

- **Node.js official download —** https://nodejs.org/en/download
- **nvm — Node Version Manager (macOS/Linux) —** https://github.com/nvm-sh/nvm
- **nvm-windows —** https://github.com/coreybutler/nvm-windows

1.2 VS Code Setup and Extensions

Visual Studio Code (VS Code) is the recommended editor for Playwright development. It has the best Playwright extension ecosystem, IntelliSense for all Playwright APIs, and an integrated terminal.

Install VS Code

Download from https://code.visualstudio.com — available for Windows, macOS, and Linux.

Essential extensions to install

Extension	What it does
Playwright Test for VS Code	Run, debug, record tests inside VS Code with a GUI
ESLint	Catches JS errors and style issues as you type
Prettier	Auto-formats your code on save
GitLens	Enhanced Git history and blame annotations
Path IntelliSense	Auto-completes file paths in import statements

Playwright VS Code extension features

- Run individual tests with a single click from the test explorer
- Step-through debugging directly inside VS Code
- Record new tests using Codegen without opening a terminal
- View test results and traces without leaving the editor

- **Playwright Test for VS Code —** https://marketplace.visualstudio.com/items?itemName=ms-playwright.playwright
- **VS Code download —** https://code.visualstudio.com

1.3 Installing Playwright

The official Playwright initialiser sets up everything you need: the test runner, configuration file, example tests, and browser binaries. Run it inside a new empty folder.

Step-by-step installation

```
# Step 1 — Create a project folder and enter it
mkdir playwright-automation
cd playwright-automation

# Step 2 — Run the official Playwright setup wizard
```

```
npm init playwright@latest

# The wizard asks:
#  Do you want to use TypeScript or JavaScript? → JavaScript
#  Where to put your end-to-end tests? → tests
#  Add a GitHub Actions workflow? → Yes (recommended)
#  Install Playwright browsers? → Yes
```

NOTE

What gets installed

Playwright downloads full browser binaries for Chromium (~120 MB), Firefox (~80 MB), and WebKit (~60 MB). These are stored in your home directory (~/.cache/ms-playwright) and shared across projects.

Install browsers separately if needed

```
# Install all browsers (Chromium, Firefox, WebKit)
npx playwright install

# Install specific browser only
npx playwright install chromium
npx playwright install firefox

# Install with system dependencies (needed in CI/Linux)
npx playwright install --with-deps

# Check which browsers are installed
npx playwright --version
```

Verify installation

```
# Run the example tests that come with the setup
npx playwright test

# Expected output:
# Running 6 tests using 3 workers
# 6 passed (10s)

# Open the HTML report
npx playwright show-report
```

- **playwright.dev — Installation guide —** https://playwright.dev/docs/intro
- **playwright.dev — Browsers guide —** https://playwright.dev/docs/browsers

1.4 Project Structure Explained

After running npm init playwright@latest, your project has this structure. Understanding every file helps you navigate the framework confidently.

```
playwright-automation/
  tests/
    example.spec.js          # sample test — delete or keep as reference
  tests-examples/
    demo-todo-app.spec.js    # larger example — good for learning
  playwright.config.js       # IMPORTANT: all configuration lives here
  package.json               # project metadata and npm scripts
  package-lock.json          # locked dependency versions
  .gitignore                 # tells Git to ignore node_modules etc.
  node_modules/              # installed packages (never edit manually)

  # Created after first test run:
  test-results/              # raw test output (screenshots, traces, videos)
  playwright-report/         # HTML report
```

Recommended structure for a real project

```
playwright-automation/
  pages/                     # Page Object Model classes
    LoginPage.js
    DashboardPage.js
    CheckoutPage.js
  tests/
    auth/
      login.spec.js
      logout.spec.js
    checkout/
      purchase-flow.spec.js
    api/
      users-api.spec.js
  fixtures/                  # Custom Playwright fixtures
    auth-fixture.js
  data/                      # Test data (JSON files)
    users.json
    products.json
  utils/                     # Helper functions
    date-helpers.js
  playwright.config.js
  package.json
```

1.5 playwright.config.js Deep-Dive

The config file controls every aspect of how tests run. Understanding it is essential for running tests in CI, across multiple browsers, and with the right timeouts.

```
// playwright.config.js — fully annotated
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
```

```
  // Directory where test files are located
  testDir: './tests',

  // Run all tests in parallel (default: true)
  fullyParallel: true,

  // Fail build on CI if test.only() is left in code
  forbidOnly: !!process.env.CI,

  // Retry failing tests 2 times in CI, 0 locally
  retries: process.env.CI ? 2 : 0,

  // Number of parallel workers
  // In CI use 1 to avoid resource contention
  workers: process.env.CI ? 1 : undefined,

  // Test reporter
  reporter: 'html',

  // Shared settings for every test
  use: {
    // Base URL for page.goto('/path') shorthand
    baseURL: 'https://www.saucedemo.com',

    // Capture trace on first retry of a failed test
    trace: 'on-first-retry',

    // Take screenshot only on failure
    screenshot: 'only-on-failure',

    // Record video only on failure
    video: 'retain-on-failure',

    // Global timeout for each action (ms)
    actionTimeout: 10000,

    // Global navigation timeout (ms)
    navigationTimeout: 30000,
  },

  // Run tests in multiple browsers
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
    // Mobile emulation
    {
```

```
      name: 'mobile-chrome',
      use: { ...devices['Pixel 7'] },
    },
  ],

  // Global test timeout in ms (default: 30000)
  timeout: 30000,

  // Expect assertion timeout
  expect: {
    timeout: 5000
  },
});
```

- **playwright.dev — Configuration reference —** https://playwright.dev/docs/test-configuration

SECTION 2 — JAVASCRIPT ESSENTIALS FOR PLAYWRIGHT

JavaScript Essentials for Playwright

Playwright is pure JavaScript/TypeScript. You don't need to master the entire language — but you must be completely comfortable with the following concepts before Day 1. Every single Playwright test uses async/await. Without understanding it, you will not be able to debug or modify tests.

2.1 let, const, and var

JavaScript has three ways to declare variables. In modern JS (and in all Playwright code), you should only use let and const.

```
// const — value cannot be reassigned (use by default)
const browser = await chromium.launch();
const page = await browser.newPage();
const title = await page.title();

// let — value can be reassigned (use when you need to update)
let retries = 0;
retries = retries + 1;

// var — OLD style, function-scoped, avoid completely
// var x = 5;  // Never use this in new code
```

RULE

Always use const first

Declare every variable with const. Only switch to let if you get an assignment error. Never use var — it has confusing scoping behaviour that causes bugs.

2.2 Arrow Functions

Arrow functions are the compact function syntax used throughout Playwright code. The async ({ page }) => {} pattern you see in every test is an arrow function.

```
// Traditional function
function add(a, b) {
  return a + b;
}

// Arrow function — same thing, shorter
const add = (a, b) => a + b;
```

```
// Arrow function with body (multiple lines)
const greet = (name) => {
  const message = 'Hello ' + name;
  return message;
};

// Async arrow function — this is the Playwright test pattern
test('my test', async ({ page }) => {
  await page.goto('https://google.com');
  //           ^^^ this is an async arrow function
});
```

2.3 Destructuring and Spread

```
// Object destructuring — extract specific properties
const user = { name: 'Alice', email: 'alice@test.com', role: 'admin' };
const { name, email } = user;  // name = 'Alice', email = 'alice@test.com'

// Array destructuring — extract by position
const [first, second] = ['Chromium', 'Firefox', 'WebKit'];
// first = 'Chromium', second = 'Firefox'

// Spread — merge objects
const base = { headless: true, timeout: 30000 };
const overrides = { headless: false };
const config = { ...base, ...overrides };
// config = { headless: false, timeout: 30000 }

// In Playwright config you see this all the time:
{ ...devices['Desktop Chrome'], baseURL: 'https://myapp.com' }
```

2.4 Promises

Asynchronous operations in JavaScript return Promises — objects representing a future value. Any browser action (clicking, navigating, typing) is asynchronous because it involves waiting for the browser to respond.

```
// A Promise represents a value that will be available later
const promise = page.goto('https://google.com');
// At this point, the navigation has STARTED but not finished

// .then() runs when the promise resolves (succeeds)
promise.then(response => {
  console.log('Status:', response.status());
});

// .catch() runs when the promise rejects (fails)
promise.catch(error => {
  console.error('Navigation failed:', error.message);
```

```
});

// Chaining multiple async operations
page.goto('/login')
  .then(() => page.fill('#email', 'user@test.com'))
  .then(() => page.fill('#password', 'secret'))
  .then(() => page.click('#submit'));
// This is hard to read — use async/await instead
```

NOTE

You rarely write .then() in Playwright

The async/await syntax is just cleaner syntax for working with Promises. Under the hood they are the same. Understanding Promises helps you debug errors like 'UnhandledPromiseRejectionWarning'.

2.5 async / await — The Critical Pattern

This is the single most important JavaScript concept for Playwright. Every test function is async. Every browser action must be awaited. If you forget await, your test will pass incorrectly or throw cryptic errors.

```
// The async keyword marks a function as asynchronous
// It means the function returns a Promise
async function runTest() {

  // await PAUSES execution until the Promise resolves
  // You MUST await every Playwright action
  await page.goto('https://google.com');     // wait for navigation
  await page.getByLabel('Search').fill('Playwright'); // wait for fill
  await page.getByLabel('Search').press('Enter');     // wait for keypress

  // await also unwraps the resolved value
  const title = await page.title();          // title is a string
  const url   = await page.url();            // url is a string
  const count = await page.locator('li').count(); // count is a number

  console.log(title, url, count);
}
```

What happens when you forget await

```
// WRONG — missing await
test('bad test', async ({ page }) => {
  page.goto('https://google.com');  // no await — navigation starts but test
doesn't wait
  const title = page.title();       // title is a Promise object, NOT a string!
  console.log(title);               // prints: Promise { <pending> }
});

// CORRECT — with await
test('good test', async ({ page }) => {
```

```
  await page.goto('https://google.com');   // waits for full load
  const title = await page.title();        // waits and gets string
  console.log(title);                      // prints: 'Google'
});
```

2.6 try / catch Error Handling

When an awaited operation fails, it throws an error. Use try/catch to handle errors gracefully — especially useful when dealing with optional elements or flaky network conditions.

```
test('handle optional elements', async ({ page }) => {
  await page.goto('https://example.com');

  // Handle an element that may or may not exist
  try {
    const cookieBanner = page.getByRole('button', { name: 'Accept Cookies' });
    await cookieBanner.click({ timeout: 3000 }); // wait max 3 seconds
    console.log('Cookie banner accepted');
  } catch (error) {
    console.log('No cookie banner — continuing...');
    // Test continues normally
  }

  // Now proceed with the actual test
  await expect(page.getByRole('heading')).toBeVisible();
});
```

- **MDN — JavaScript Guide —** https://developer.mozilla.org/en-US/docs/Web/JavaScript
- **javascript.info — async/await —** https://javascript.info/async-await
- **javascript.info — Promises —** https://javascript.info/promise-basics
- **javascript.info — Arrow functions —** https://javascript.info/arrow-functions-basics

SECTION 3 — WEEK 1: CORE FOUNDATION

Week 1: Core Foundation

GOAL

Week 1 objective

By the end of this week you can install Playwright, write tests from scratch, navigate websites, interact with elements, and use Playwright's locator system confidently.

DAY 1 – 2

JavaScript Essentials Review + Practice Exercises

Days 1 and 2 are dedicated to JavaScript — specifically the async/await concepts from Section 2. Read through Section 2 carefully, then complete these exercises before moving forward. Skipping this step causes confusion on Day 3.

Exercise 1 — Async/await chain

Write an async function that performs three operations in sequence: fetch a URL, parse the response as JSON, and log the first item's name. Use async/await throughout.

```
// Exercise 1 — Complete this function
async function fetchFirstUser() {
  const response = await fetch('https://reqres.in/api/users');
  const data     = await response.json();
  const firstUser = data.data[0];
  console.log('First user:', firstUser.first_name, firstUser.last_name);
  // Expected output: First user: George Bluth
}

fetchFirstUser();
```

Exercise 2 — Error handling

Modify the above function to handle network errors gracefully. If the fetch fails, log the error and return null instead of crashing.

```
// Exercise 2 — Add error handling
async function fetchFirstUserSafe() {
  try {
    const response = await fetch('https://reqres.in/api/users');
    if (!response.ok) throw new Error(`HTTP error: ${response.status}`);
    const data = await response.json();
    return data.data[0];
  } catch (error) {
```

```
    console.error('Fetch failed:', error.message);
    return null;
  }
}
```

Exercise 3 — Arrow functions and destructuring

```
// Rewrite using arrow functions and destructuring
const users = [
  { name: 'Alice', role: 'admin' },
  { name: 'Bob',   role: 'viewer' },
  { name: 'Carol', role: 'admin' },
];

// Get all admin names using arrow function + destructuring
const admins = users
  .filter(({ role }) => role === 'admin')
  .map(({ name }) => name);

console.log(admins); // ['Alice', 'Carol']
```

Day 1–2 resources

- **javascript.info — Modern JavaScript Tutorial —** https://javascript.info
- **MDN — async function —** https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
- **MDN — Promises —** https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

DAY 3 — Your First Playwright Tests — Setup to Running

Day 3 is all about running real Playwright tests for the first time. Follow these steps exactly. By the end of today, you will have Playwright installed, have run the example tests, and have written and run your own first test.

Step 1: Create the project

```
mkdir my-playwright-project
cd my-playwright-project
npm init playwright@latest

# Answer the prompts:
# ? Do you want to use TypeScript or JavaScript? › JavaScript
# ? Where to put your end-to-end tests? › tests
# ? Add a GitHub Actions workflow? › true
# ? Install Playwright browsers (can be done manually via 'npx playwright
install')? › true
```

Step 2: Explore the generated example test

Open tests/example.spec.js. Read it carefully. Notice: every test function is async, every action uses await, and tests are grouped with test.describe().

```
// tests/example.spec.js — the generated file
import { test, expect } from '@playwright/test';

test('has title', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  await expect(page).toHaveTitle(/Playwright/);
});

test('get started link', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  await page.getByRole('link', { name: 'Get started' }).click();
  await expect(page.getByRole('heading', { name: 'Installation'
})).toBeVisible();
});
```

Step 3: Run the tests

```
# Run all tests (headless by default — no browser window)
npx playwright test

# Run with visible browser windows
npx playwright test --headed

# Run in interactive UI mode (recommended for learning)
npx playwright test --ui

# Run only one test file
npx playwright test tests/example.spec.js

# Run tests matching a name pattern
npx playwright test -g 'has title'

# Run in a specific browser
npx playwright test --project=firefox

# Open the HTML report after a run
npx playwright show-report
```

Step 4: Write your own first test

Delete the contents of tests/example.spec.js and write this test from scratch. Type it — do not copy-paste.

```
// tests/my-first.spec.js
import { test, expect } from '@playwright/test';
```

```
test.describe('My first tests', () => {

  test('Google has the right title', async ({ page }) => {
    // Navigate to Google
    await page.goto('https://google.com');

    // Get and log the page title
    const title = await page.title();
    console.log('Page title:', title);

    // Assert the title contains 'Google'
    await expect(page).toHaveTitle(/Google/);
  });

  test('Playwright docs site loads', async ({ page }) => {
    await page.goto('https://playwright.dev/');
    await expect(page).toHaveTitle(/Playwright/);
    await expect(page).toHaveURL('https://playwright.dev/');
  });

  test('Can navigate to a second page', async ({ page }) => {
    await page.goto('https://playwright.dev/');

    // Click the Docs link
    await page.getByRole('link', { name: 'Docs' }).first().click();

    // URL should change
    await expect(page).not.toHaveURL('https://playwright.dev/');
    console.log('Navigated to:', page.url());
  });

});
```

TIP

test.describe() grouping

Use test.describe() to group related tests. It appears as a collapsible section in the HTML report and lets you run an entire group with: npx playwright test -g 'My first tests'

- **playwright.dev — Writing tests —** https://playwright.dev/docs/writing-tests
- **playwright.dev — Command line options —** https://playwright.dev/docs/test-cli

DAY 4 – 5

Navigation and Page Actions — Complete Reference

Days 4 and 5 cover all the fundamental browser actions. Playwright's page object has methods for everything a real user can do: navigate, click, type, press keys, hover, drag, and more.

Navigation methods

```
// Navigate to a URL — waits for load event by default
await page.goto('https://example.com');

// Navigate with specific wait conditions
await page.goto('https://example.com', { waitUntil: 'domcontentloaded' });
await page.goto('https://example.com', { waitUntil: 'networkidle' });
await page.goto('https://example.com', { waitUntil: 'load' }); // default

// With a baseURL set in config, you can use relative paths
await page.goto('/login');        // goes to baseURL + /login
await page.goto('/dashboard');    // goes to baseURL + /dashboard

// Browser navigation
await page.goBack();
await page.goForward();
await page.reload();

// Get current URL and title
const url   = page.url();         // synchronous — no await needed
const title = await page.title(); // async
```

Clicking

```
// Standard click
await page.getByRole('button', { name: 'Submit' }).click();

// Double click
await page.getByText('Edit').dblclick();

// Right click (context menu)
await page.getByText('File').click({ button: 'right' });

// Click with modifiers
await page.getByRole('link', { name: 'Home' }).click({ modifiers: ['Meta'] });
// Cmd+Click (macOS)
await page.getByRole('link', { name: 'Home' }).click({ modifiers: ['Control']
}); // Ctrl+Click

// Click at specific coordinates within element
await page.getByRole('canvas').click({ position: { x: 100, y: 150 } });

// Force click — bypasses actionability checks
// Use ONLY when element is obscured but you know it is clickable
await page.locator('#hidden-btn').click({ force: true });
```

Typing and keyboard

```
// fill() — clears the field and types (best for most inputs)
await page.getByLabel('Email').fill('user@test.com');

// type() — types character-by-character with delay (use when fill() fails)
await page.getByLabel('Search').type('playwright', { delay: 100 });
```

```
// clear() — clear an input without typing
await page.getByLabel('Email').clear();

// press() — keyboard keys
await page.getByLabel('Search').press('Enter');
await page.getByLabel('Name').press('Tab');     // move to next field
await page.keyboard.press('Escape');             // global keypress
await page.keyboard.press('Control+A');          // Ctrl+A to select all
await page.keyboard.press('Control+C');          // Ctrl+C to copy

// Type then press Enter — common pattern
await page.getByRole('searchbox').fill('test query');
await page.getByRole('searchbox').press('Enter');
```

Mouse and hover

```
// Hover over element (triggers tooltip, dropdown, etc.)
await page.getByRole('button', { name: 'Account' }).hover();

// Wait for the hover-triggered element to appear
await page.getByRole('button', { name: 'Account' }).hover();
await expect(page.getByRole('menu')).toBeVisible();

// Mouse movement
await page.mouse.move(100, 200);
await page.mouse.down();
await page.mouse.move(400, 200); // drag
await page.mouse.up();

// Scroll with mouse wheel
await page.mouse.wheel(0, 500);  // scroll down 500px
```

Getting element properties

```
// Get text content of an element
const heading = await page.getByRole('heading').textContent();
const allItems = await page.getByRole('listitem').allTextContents();

// Get an attribute value
const href = await page.getByRole('link', { name: 'Home'
}).getAttribute('href');
const src  = await page.locator('img.logo').getAttribute('src');

// Get input value
const email = await page.getByLabel('Email').inputValue();

// Check visibility / enabled state
const isVisible = await page.getByRole('button').isVisible();
const isEnabled = await page.getByRole('button').isEnabled();
const isChecked = await page.getByRole('checkbox').isChecked();
```

Full example — Google Search

```
test('complete Google search flow', async ({ page }) => {
  // 1. Navigate
  await page.goto('https://www.google.com');

  // 2. Handle cookie consent (present in some regions)
  try {
    await page.getByRole('button', { name: /Accept all/i }).click({ timeout:
3000 });
  } catch { /* not present — skip */ }

  // 3. Type search query
  await page.getByRole('combobox', { name: 'Search' }).fill('Playwright
JavaScript tutorial');

  // 4. Submit
  await page.getByRole('combobox', { name: 'Search' }).press('Enter');

  // 5. Wait for results
  await expect(page).toHaveURL(/search/);

  // 6. Assert at least one result exists
  const results = page.locator('#search .g');
  await expect(results.first()).toBeVisible();
  console.log('Result count:', await results.count());

  // 7. Click first result
  await results.first().getByRole('link').first().click();
  console.log('Navigated to:', page.url());
});
```

- **playwright.dev — Page API —** https://playwright.dev/docs/api/class-page
- **playwright.dev — Keyboard API —** https://playwright.dev/docs/api/class-keyboard
- **playwright.dev — Mouse API —** https://playwright.dev/docs/api/class-mouse

DAY 6 – 7 Locators — The Most Important Skill in Playwright

CRITICAL

Why locators matter so much

Bad locators are the #1 cause of flaky tests. A flaky test is one that sometimes passes and sometimes fails for no clear reason. Playwright's locator system is designed to prevent flakiness by using user-visible attributes rather than implementation-specific CSS paths or XPath expressions.

A locator is a reference to an element on the page. Unlike a regular DOM query, a Playwright locator does not immediately find the element — it only finds the element when you call an action on it. This means if the element takes 2 seconds to appear, Playwright waits automatically.

The locator priority order

Always try to use a locator higher on this list before falling back to a lower one:

5. getByRole — matches by ARIA role and accessible name. This is the most resilient.
6. getByLabel — for form inputs, matches the associated <label> text.
7. getByPlaceholder — for inputs with placeholder text.
8. getByText — for non-interactive elements (headings, paragraphs, spans).
9. getByAltText — for images with meaningful alt text.
10. getByTitle — for elements with a title attribute.
11. getByTestId — for elements with data-testid attribute (requires team agreement).
12. locator('css') or locator('xpath') — last resort only.

getByRole — ARIA roles reference

ARIA roles describe the purpose of an element. These are the most common roles you will target:

Role	HTML element	Usage example
button	<button>	getByRole('button', { name: 'Save' })
link	<a href>	getByRole('link', { name: 'Home' })
textbox	<input type=text>	getByRole('textbox', { name: 'Email' })
checkbox	<input type=checkbox>	getByRole('checkbox', { name: 'Agree' })
radio	<input type=radio>	getByRole('radio', { name: 'Monthly' })
combobox	<select> or custom	getByRole('combobox', { name: 'Country' })
heading	<h1> to <h6>	getByRole('heading', { name: 'Dashboard' })
img	<img>	getByRole('img', { name: 'Profile photo' })
navigation	<nav>	getByRole('navigation')
main	<main>	getByRole('main')
listitem		getByRole('listitem')
dialog	<dialog>	getByRole('dialog')

getByRole — detailed examples

```
// Exact name match (default)
await page.getByRole('button', { name: 'Submit' }).click();

// Case-insensitive regex match
await page.getByRole('button', { name: /submit/i }).click();
```

```
// Exact: false — partial name match
await page.getByRole('button', { name: 'Save', exact: false }).click();

// Level for headings
await page.getByRole('heading', { level: 1 });   // <h1>
await page.getByRole('heading', { level: 2 });   // <h2>

// Checked state for checkboxes/radios
await page.getByRole('checkbox', { checked: true });  // already checked
await page.getByRole('checkbox', { checked: false }); // not checked

// Disabled state
await page.getByRole('button', { disabled: true });
```

getByLabel — form inputs

```
// HTML: <label for='email'>Email address</label>
//       <input id='email' type='email'>
await page.getByLabel('Email address').fill('user@test.com');

// HTML: <label>Password <input type='password'></label>
await page.getByLabel('Password').fill('secret123');

// Works with aria-label too
// HTML: <input aria-label='Search products'>
await page.getByLabel('Search products').fill('laptop');
```

Filtering and chaining locators

```
// Filter by text within a locator
const items = page.getByRole('listitem');
const adminItem = items.filter({ hasText: 'Admin' });
await adminItem.click();

// Filter by sub-locator
const rows = page.getByRole('row');
const activeRows = rows.filter({ has: page.getByText('Active') });

// Narrow to a section first, then find within it
const form = page.getByRole('form', { name: 'Login' });
await form.getByLabel('Email').fill('user@test.com');
await form.getByLabel('Password').fill('secret');
await form.getByRole('button', { name: 'Log in' }).click();

// nth — pick the Nth matching element
const allButtons = page.getByRole('button');
await allButtons.nth(0).click();  // first button
await allButtons.last().click();  // last button

// first() / last() shorthand
await page.getByRole('link').first().click();
```

CSS selectors — when to use them

```
// Use CSS locators ONLY when semantic locators are not available

// By ID
await page.locator('#submit-btn').click();

// By class
await page.locator('.product-card').first().click();

// By attribute
await page.locator('input[type="email"]').fill('test@test.com');
await page.locator('[data-testid="login-form"]').isVisible();

// Descendant
await page.locator('.cart-items .item-name').textContent();

// :has — parent containing a child
await page.locator('.product-card:has(.badge-sale)').first().click();

// nth-child
await page.locator('table tbody tr:nth-child(2) td:nth-
child(3)').textContent();
```

Debugging locators — finding the right selector

```
# Use Codegen to visually pick selectors
npx playwright codegen https://demoqa.com

# Use the Inspector in --ui mode
npx playwright test --ui

# Use the browser DevTools console to test locators
# Open DevTools > Console and type:
# document.querySelector('#your-selector') — to check CSS
```

PRACTICE

Day 6–7 exercises

Open demoqa.com and write tests that: (1) fill in the Practice Form under Forms, (2) click each element in the Buttons section, (3) validate text in the Web Tables. Use only getByRole and getByLabel — no CSS selectors.

- **playwright.dev — Locators guide —** https://playwright.dev/docs/locators
- **playwright.dev — Best practices for locators —** https://playwright.dev/docs/best-practices
- **DemoQA practice site —** https://demoqa.com
- **playwright.dev — Other locators —** https://playwright.dev/docs/other-locators

SECTION 4 — WEEK 2: REAL AUTOMATION SKILLS

Week 2: Real Automation Skills

GOAL

Week 2 objective

By the end of this week you can handle any real-world UI pattern — dropdowns, iframes, file uploads, multiple tabs — write robust assertions, wait for dynamic content correctly, and debug failing tests with the Trace Viewer.

DAY 8 – 9

Assertions and Auto-Waiting — Complete Reference

Playwright's assertion library is called expect(). It wraps elements and pages with web-first assertions that automatically retry. This means if an assertion fails, Playwright re-checks it every 100ms until it passes or the timeout is reached.

How auto-waiting works

```
// This assertion...
await expect(page.getByText('Welcome')).toBeVisible();

// ...internally does this:
//   1. Find the element
//   2. Check if it is visible
//   3. If NOT visible: wait 100ms and try again
//   4. Repeat until visible OR until timeout (default 5000ms)

// This is why you should NEVER do this:
await page.waitForTimeout(3000);       // BAD — unconditional sleep
const isVisible = await el.isVisible(); // now check

// Instead just do this:
await expect(el).toBeVisible();        // GOOD — auto-waits
```

Page-level assertions

```
// Title
await expect(page).toHaveTitle('Dashboard — MyApp');
await expect(page).toHaveTitle(/Dashboard/);      // regex

// URL
```

```
await expect(page).toHaveURL('https://app.com/dashboard');
await expect(page).toHaveURL(/\/dashboard/);       // regex

// Custom timeout (override default 5000ms)
await expect(page).toHaveURL(/dashboard/, { timeout: 10000 });
```

Element visibility and state

```
const el = page.getByRole('button', { name: 'Save' });

// Visibility
await expect(el).toBeVisible();
await expect(el).not.toBeVisible();
await expect(el).toBeHidden();

// Enabled / disabled
await expect(el).toBeEnabled();
await expect(el).toBeDisabled();

// Checked (for checkboxes and radio buttons)
await expect(page.getByRole('checkbox')).toBeChecked();
await expect(page.getByRole('checkbox')).not.toBeChecked();

// Focused
await expect(page.getByLabel('Email')).toBeFocused();

// In the DOM (attached, even if invisible)
await expect(el).toBeAttached();
```

Text content assertions

```
const heading = page.getByRole('heading');

// Exact text match
await expect(heading).toHaveText('Welcome back, Alice');

// Partial match
await expect(heading).toContainText('Welcome');

// Regex match
await expect(heading).toHaveText(/Welcome back, .+/);

// Multiple elements — each must match
await expect(page.getByRole('listitem'))
  .toHaveText(['First item', 'Second item', 'Third item']);

// Contains text (partial match for each)
await expect(page.getByRole('listitem'))
  .toContainText(['First', 'Second']);
```

Input value and attribute assertions

```
// Input value
await expect(page.getByLabel('Email')).toHaveValue('user@test.com');
await expect(page.getByLabel('Email')).toHaveValue(/test\.com/);

// Attribute
await expect(page.locator('img.logo')).toHaveAttribute('alt', 'Company logo');
await expect(page.getByRole('link', { name: 'Home' }))
  .toHaveAttribute('href', '/');

// CSS class
await expect(page.locator('.status-badge')).toHaveClass(/active/);
await expect(page.locator('.status-badge')).toHaveClass('badge badge-active');

// Count
await expect(page.getByRole('listitem')).toHaveCount(5);

// CSS property value
await expect(page.locator('.hero')).toHaveCSS('color', 'rgb(0, 0, 0)');
```

Soft assertions — continue after failure

```
// Regular assertion — stops the test immediately on failure
await expect(page).toHaveTitle('Dashboard');

// Soft assertion — records failure but test continues
await expect.soft(page).toHaveTitle('Dashboard');
await expect.soft(page.getByText('Welcome')).toBeVisible();
await expect.soft(page.getByRole('button')).toBeEnabled();

// All soft assertion failures are reported at end of test
// Use when you want to check multiple things in one pass
```

Configuring assertion timeouts

```
// Per-assertion timeout
await expect(page.getByText('Loading...')).toBeHidden({ timeout: 15000 });

// Global assertion timeout in playwright.config.js
expect: {
  timeout: 10000  // 10 seconds for all assertions
},
```

- **playwright.dev — Assertions reference —** https://playwright.dev/docs/test-assertions
- **playwright.dev — Auto-waiting and actionability —** https://playwright.dev/docs/actionability

DAY 10 – 11

Advanced UI Handling — Dropdowns, iFrames, Uploads, Tabs, Dialogs

Dropdowns — <select> elements

```
// Select by value attribute
await page.selectOption('select#country', 'IN');

// Select by visible label text
await page.selectOption('select#country', { label: 'India' });

// Select by index (0-based)
await page.selectOption('select#size', { index: 2 });

// Multiple selections (for multi-select elements)
await page.selectOption('select#languages', ['js', 'python', 'java']);

// Using locator
await page.getByRole('combobox', { name: 'Country' }).selectOption('India');

// Assert selected value
await expect(page.getByRole('combobox', { name: 'Country' }))
  .toHaveValue('IN');
```

Custom dropdowns (non-<select>)

Many modern apps use custom dropdown components built with divs and spans, not native <select>. Handle these by clicking the trigger, then clicking the option:

```
// Click the dropdown trigger to open it
await page.getByRole('button', { name: 'Select Country' }).click();

// Wait for the options list to appear, then click the option
await page.getByRole('option', { name: 'India' }).click();

// Or using text if not an option role
await page.getByText('India').click();

// Assert the dropdown shows the selected value
await expect(page.getByRole('button', { name: 'Select Country' }))
  .toContainText('India');
```

Checkboxes and radio buttons

```
// Check a checkbox
await page.getByRole('checkbox', { name: 'I agree to the terms' }).check();

// Uncheck
await page.getByRole('checkbox', { name: 'Newsletter' }).uncheck();
```

```
// Set checked state regardless of current state
await page.getByRole('checkbox', { name: 'Agree' }).setChecked(true);
await page.getByRole('checkbox', { name: 'Agree' }).setChecked(false);

// Assert checked state
await expect(page.getByRole('checkbox', { name: 'Agree' })).toBeChecked();

// Radio buttons
await page.getByRole('radio', { name: 'Monthly billing' }).check();
await expect(page.getByRole('radio', { name: 'Monthly billing'
})).toBeChecked();
```

iFrames — complete guide

iFrames are embedded HTML documents inside a page. To interact with content inside an iframe, you must use frameLocator() to get a reference to the iframe's context.

```
// Access iframe by CSS selector
const frame = page.frameLocator('iframe#payment-frame');

// Access by name attribute
const frame2 = page.frameLocator('iframe[name="payment"]');

// Interact with elements inside the iframe just like normal
await frame.getByLabel('Card number').fill('4111 1111 1111 1111');
await frame.getByLabel('Expiry date').fill('12/28');
await frame.getByLabel('CVV').fill('123');
await frame.getByRole('button', { name: 'Pay' }).click();

// Nested iframes
const innerFrame = page
  .frameLocator('iframe#outer')
  .frameLocator('iframe#inner');

await innerFrame.getByRole('button').click();

// Assert content inside iframe
await expect(frame.getByText('Payment successful')).toBeVisible();
```

File upload

```
// Single file upload
await page.setInputFiles('input[type="file"]', './files/document.pdf');

// Using getByLabel if the input has a label
await page.getByLabel('Upload CV').setInputFiles('./resume.pdf');

// Multiple files
await page.setInputFiles('input[type="file"]', [
  './files/photo1.png',
  './files/photo2.png',
```

```
  './files/document.pdf',
]);

// Clear file selection
await page.setInputFiles('input[type="file"]', []);

// Drag-and-drop file upload (for custom drop zones)
await page.locator('#drop-zone').dispatchEvent('drop', {
  dataTransfer: await page.evaluateHandle(() => {
    const dt = new DataTransfer();
    return dt;
  })
});

// Assert file name appeared
await expect(page.getByText('document.pdf')).toBeVisible();
```

Multiple tabs and windows

```
// Wait for a new tab to open when clicking a link
const [newPage] = await Promise.all([
  page.context().waitForEvent('page'),    // listen for new page
  page.getByRole('link', { name: 'Open in new tab' }).click()
]);

// Wait for the new tab to fully load
await newPage.waitForLoadState('domcontentloaded');

// Interact with the new tab
await expect(newPage).toHaveTitle(/External Site/);
const result = await newPage.getByRole('heading').textContent();

// Close the tab
await newPage.close();

// You are now back on the original page
await expect(page).toHaveURL(/original-page/);

// List all open pages
const pages = page.context().pages();
console.log('Open tabs:', pages.length);
```

Handling dialogs — alert, confirm, prompt

```
// Accept all dialogs automatically
page.on('dialog', async dialog => {
  console.log('Dialog type:', dialog.type());    // alert, confirm, prompt
  console.log('Dialog message:', dialog.message());
  await dialog.accept();
});

// Dismiss (cancel) a confirm dialog
```

```
page.on('dialog', async dialog => {
  if (dialog.type() === 'confirm') {
    await dialog.dismiss();
  } else {
    await dialog.accept();
  }
});

// Type into a prompt dialog
page.on('dialog', async dialog => {
  await dialog.accept('My typed answer');
});

// Trigger the action that causes the dialog
await page.getByRole('button', { name: 'Delete' }).click();
```

- **playwright.dev — iFrames —** https://playwright.dev/docs/frames
- **playwright.dev — Multiple pages (tabs) —** https://playwright.dev/docs/pages
- **playwright.dev — File upload —** https://playwright.dev/docs/input#upload-files
- **playwright.dev — Dialogs —** https://playwright.dev/docs/dialogs
- **The Internet — practice all UI patterns —** https://the-internet.herokuapp.com

DAY 12 Scrolling, Lazy Loading, and Dynamic Content

Modern web applications load content dynamically — infinite scroll, lazy-loaded images, skeleton loaders, and real-time data updates. This section covers all patterns for handling dynamic content reliably.

Scrolling methods

```
// Scroll the entire page with mouse wheel
await page.mouse.wheel(0, 800);    // down 800px
await page.mouse.wheel(0, -400);   // up 400px

// Scroll to the very bottom of the page
await page.evaluate(() => window.scrollTo(0, document.body.scrollHeight));

// Scroll to a specific Y position
await page.evaluate(() => window.scrollTo(0, 2000));

// Scroll an element into the visible area
await page.locator('.footer').scrollIntoViewIfNeeded();
await page.getByText('Load more').scrollIntoViewIfNeeded();

// Scroll inside a scrollable container (not the whole page)
const container = page.locator('#scroll-container');
await container.evaluate(el => el.scrollTop = el.scrollHeight);
```

Waiting for dynamic content

```
// Wait for a specific element to appear in the DOM
await page.waitForSelector('.search-results');

// Better: use assertions which auto-retry
await expect(page.locator('.search-results')).toBeVisible();

// Wait for element count to be N
await expect(page.getByRole('listitem')).toHaveCount(10);

// Wait for an element to disappear
await expect(page.locator('.loading-spinner')).toBeHidden();

// Wait for text to change
await expect(page.getByTestId('status')).toHaveText('Completed');

// Wait for URL change (after form submit)
await Promise.all([
  page.waitForURL(/\/success/),
  page.getByRole('button', { name: 'Submit' }).click()
]);
```

Network waiting

```
// Wait for a specific API response before proceeding
const responsePromise = page.waitForResponse('**/api/products');
await page.getByRole('button', { name: 'Load products' }).click();
const response = await responsePromise;
expect(response.status()).toBe(200);

// Wait for navigation AND response together
await Promise.all([
  page.waitForResponse(res => res.url().includes('/api/search')
    && res.status() === 200),
  page.getByRole('button', { name: 'Search' }).click()
]);

// Wait for the network to be idle (no requests for 500ms)
await page.waitForLoadState('networkidle');
// Note: use sparingly — only for pages with known network patterns
```

Infinite scroll pattern

```
test('load all items via infinite scroll', async ({ page }) => {
  await page.goto('https://infinite-scroll-demo.com');

  let previousCount = 0;
  let currentCount = await page.getByRole('listitem').count();

  // Keep scrolling until no new items load
  while (currentCount > previousCount) {
```

```
    previousCount = currentCount;

    // Scroll to bottom
    await page.evaluate(() => window.scrollTo(0, document.body.scrollHeight));

    // Wait for potential new content
    await page.waitForTimeout(1500); // acceptable here — wait for load

    currentCount = await page.getByRole('listitem').count();
    console.log('Items loaded:', currentCount);
  }

  console.log('Total items:', currentCount);
  await expect(page.getByRole('listitem')).toHaveCount(currentCount);
});
```

- **playwright.dev — Network —** https://playwright.dev/docs/network
- **playwright.dev — Navigations —** https://playwright.dev/docs/navigations

DAY 13 – 14 Debugging — Trace Viewer, Codegen, UI Mode, Screenshots

Debugging is where most beginners lose hours. Playwright's debugging tools — especially the Trace Viewer — are world-class. Learn them on Day 13 and use them every time a test fails, for the rest of your career.

The Trace Viewer — your most important debugging tool

The Trace Viewer records every action in a test run and lets you play it back step-by-step. For each step, it shows: (1) a screenshot of the browser, (2) the action performed, (3) the DOM before and after, and (4) network requests.

```
# Run tests with tracing enabled
npx playwright test --trace on

# Open a specific trace file
npx playwright show-trace test-results/my-test/trace.zip

# Upload a trace to the online viewer (no install needed)
# Go to: https://trace.playwright.dev
# Drag and drop your trace.zip file
```

Trace configuration in playwright.config.js

```
use: {
  // 'off'              — never record
  // 'on'               — always record
```

```
  // 'retain-on-failure' — record, delete if passes
  // 'on-first-retry'   — record only when retrying (best for CI)
  trace: 'on-first-retry',
},
```

Codegen — record tests by interacting

Codegen opens a browser and records every click, fill, and navigation as Playwright code. Use it to bootstrap selectors quickly.

```
# Start codegen for a URL
npx playwright codegen https://demoqa.com

# Save generated code to a file
npx playwright codegen https://demoqa.com --output tests/recorded.spec.js

# Emulate a specific device
npx playwright codegen --device='iPhone 13' https://example.com

# Set the viewport
npx playwright codegen --viewport-size='1280,720' https://example.com
```

TIP

Codegen generates starting code only

The code produced by Codegen is a starting point. Always refactor: replace brittle CSS locators with getByRole/getByLabel, remove unnecessary waitForTimeout calls, and add proper assertions.

UI Mode — interactive test runner

```
# Launch UI mode
npx playwright test --ui

# UI mode features:
# - Run individual tests with one click
# - Watch mode — auto-runs tests on file save
# - Step through tests action by action
# - View screenshots for each step
# - Inspect locators live
# - Time-travel debugging
```

Screenshots and videos

```
// Full-page screenshot in a test
await page.screenshot({
  path: 'screenshots/test-state.png',
  fullPage: true   // capture entire scrollable page
});

// Screenshot of a specific element
await page.locator('.product-card').screenshot({
```

```
  path: 'screenshots/product.png'
});

// Return screenshot as Buffer (for assertions)
const screenshot = await page.screenshot();
expect(screenshot).toBeTruthy(); // basic existence check

// Configure screenshots in playwright.config.js
use: {
  screenshot: 'only-on-failure',  // 'on', 'off', 'only-on-failure'
  video: 'retain-on-failure',     // 'on', 'off', 'retain-on-failure'
},
```

Debugging with console.log and page.evaluate

```
// Log browser console messages in Node
page.on('console', msg => {
  console.log('[Browser]', msg.type(), msg.text());
});

// Run JavaScript in the browser context
const localStorageData = await page.evaluate(() => {
  return JSON.parse(localStorage.getItem('user') || '{}');
});
console.log('LocalStorage user:', localStorageData);

// Get element count for debugging
const count = await page.locator('.item').count();
console.log('Items found:', count);

// Pause test execution (opens Playwright Inspector)
await page.pause(); // REMOVE before committing!
```

- **playwright.dev — Trace Viewer —** https://playwright.dev/docs/trace-viewer
- **playwright.dev — Codegen —** https://playwright.dev/docs/codegen
- **playwright.dev — UI Mode —** https://playwright.dev/docs/test-ui-mode
- **Online trace viewer —** https://trace.playwright.dev

SECTION 5 — WEEK 3: FRAMEWORK & ADVANCED PATTERNS

Week 3: Framework & Advanced Patterns

GOAL

Week 3 objective

Build a fully maintainable test automation framework from scratch — with Page Object Model, custom fixtures, data-driven tests, API testing, and CI/CD pipeline.

DAY 15 – 16

Fixtures and Test Structure

Fixtures are Playwright's most powerful mechanism for test organisation and code reuse. They replace beforeEach/afterEach blocks with a cleaner, composable system. Understanding fixtures deeply separates beginner Playwright users from advanced ones.

Built-in fixtures

Playwright provides several built-in fixtures that are injected into every test. You use them as parameters in the test function:

Fixture	What it provides
page	A fresh browser page (tab) for each test
browser	The browser instance — shared across tests
browserName	Current browser as string: chromium / firefox / webkit
context	The browser context — isolated cookie/session store
request	API request context for HTTP testing

```
// Using built-in fixtures
test('check browser name', async ({ page, browserName }) => {
  console.log('Running on:', browserName); // 'chromium' or 'firefox'
  await page.goto('https://example.com');
});

test('make an API request', async ({ request }) => {
  const res = await request.get('https://reqres.in/api/users');
  expect(res.status()).toBe(200);
});
```

beforeEach, afterEach, beforeAll, afterAll

```
import { test, expect } from '@playwright/test';

test.describe('User dashboard tests', () => {

  // Runs before EVERY test in this describe block
  test.beforeEach(async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill('admin@test.com');
    await page.getByLabel('Password').fill('secret');
    await page.getByRole('button', { name: 'Log in' }).click();
    await expect(page).toHaveURL('/dashboard');
  });

  // Runs after EVERY test
  test.afterEach(async ({ page }, testInfo) => {
    if (testInfo.status !== testInfo.expectedStatus) {
      await page.screenshot({ path: `failed-${testInfo.title}.png` });
    }
  });

  // Runs ONCE before all tests in this block
  test.beforeAll(async () => {
    console.log('Setting up test data...');
  });

  // Runs ONCE after all tests in this block
  test.afterAll(async () => {
    console.log('Cleaning up...');
  });

  test('dashboard shows username', async ({ page }) => {
    await expect(page.getByText('Admin')).toBeVisible();
  });

  test('dashboard shows menu items', async ({ page }) => {
    await expect(page.getByRole('navigation')).toBeVisible();
  });

});
```

Custom fixtures — the right approach for complex setup

Custom fixtures let you package reusable setup/teardown logic into named, injectable units. Any test that needs them just declares them as parameters.

```
// fixtures/auth-fixture.js
import { test as base, expect } from '@playwright/test';

// Extend the base test with custom fixtures
export const test = base.extend({

  // Fixture: authenticated page — auto-logs in before test
  authenticatedPage: async ({ page }, use) => {
```

```
    // SETUP — runs before the test
    await page.goto('/login');
    await page.getByLabel('Email').fill('admin@test.com');
    await page.getByLabel('Password').fill('secret123');
    await page.getByRole('button', { name: 'Log in' }).click();
    await expect(page).toHaveURL('/dashboard');

    // Yield the page to the test
    await use(page);

    // TEARDOWN — runs after the test
    await page.evaluate(() => localStorage.clear());
    await page.evaluate(() => sessionStorage.clear());
  },

  // Fixture: admin user data
  adminUser: async ({}, use) => {
    const user = { email: 'admin@test.com', password: 'secret123', role:
'admin' };
    await use(user);
  },

  // Fixture: API client
  apiClient: async ({ request }, use) => {
    // Get auth token via API
    const res = await request.post('/api/auth/login', {
      data: { email: 'admin@test.com', password: 'secret123' }
    });
    const { token } = await res.json();

    // Return configured client
    await use({ request, token });

    // Cleanup: invalidate token
    await request.post('/api/auth/logout', {
      headers: { Authorization: `Bearer ${token}` }
    });
  },

});

// Re-export expect unchanged
export { expect } from '@playwright/test';
```

Using custom fixtures in tests

```
// tests/dashboard.spec.js
// Import from your fixture file instead of @playwright/test
import { test, expect } from '../fixtures/auth-fixture';

test('dashboard loads correctly', async ({ authenticatedPage }) => {
  // Page is already logged in!
  await expect(authenticatedPage.getByRole('heading', { level: 1 }))
    .toHaveText('Dashboard');
});
```

```
test('profile shows user email', async ({ authenticatedPage, adminUser }) => {
  await authenticatedPage.getByRole('link', { name: 'Profile' }).click();
  await expect(authenticatedPage.getByText(adminUser.email)).toBeVisible();
});

test('API returns user list', async ({ apiClient }) => {
  const { request, token } = apiClient;
  const res = await request.get('/api/users', {
    headers: { Authorization: `Bearer ${token}` }
  });
  expect(res.status()).toBe(200);
});
```

- **playwright.dev — Fixtures —** https://playwright.dev/docs/test-fixtures
- **playwright.dev — Test organisation —** https://playwright.dev/docs/test-annotations

DAY 17 – 18 Page Object Model — Design, Structure, and Best Practices

The Page Object Model (POM) is the most widely used design pattern in test automation. The core principle: each page (or major component) of your application has its own class that encapsulates all selectors and interactions for that page.

Why POM is essential

- When a selector changes, you update it in ONE place instead of every test that uses it
- Tests read like user stories, not implementation details — easier for non-technical teammates to understand
- Page methods can be reused across many tests without copy-pasting
- Makes refactoring safe — tests don't break when you reorganise your POM
- Onboarding new team members is faster — the POM is a catalogue of all app interactions

Basic POM — LoginPage

```
// pages/LoginPage.js
export class LoginPage {

  constructor(page) {
    this.page = page;

    // Define all locators as class properties
    this.emailInput    = page.getByLabel('Email');
    this.passwordInput = page.getByLabel('Password');
    this.loginButton   = page.getByRole('button', { name: 'Log in' });
    this.errorMessage  = page.locator('[data-testid="error-message"]');
    this.forgotLink    = page.getByRole('link', { name: 'Forgot password?' });
```

```
  }

  // Navigation
  async goto() {
    await this.page.goto('/login');
  }

  // Core action
  async login(email, password) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }

  // Wait for login to complete successfully
  async loginSuccessfully(email, password) {
    await this.login(email, password);
    await this.page.waitForURL('/dashboard');
  }

  // Getters for test assertions
  async getErrorMessage() {
    return await this.errorMessage.textContent();
  }

  async isErrorVisible() {
    return await this.errorMessage.isVisible();
  }
}
```

DashboardPage

```
// pages/DashboardPage.js
export class DashboardPage {

  constructor(page) {
    this.page       = page;
    this.heading    = page.getByRole('heading', { level: 1 });
    this.navMenu    = page.getByRole('navigation');
    this.logoutBtn  = page.getByRole('button', { name: 'Log out' });
    this.userBadge  = page.getByTestId('user-badge');
    this.searchBox  = page.getByRole('searchbox');
  }

  async isLoaded() {
    await expect(this.heading).toBeVisible();
    await expect(this.navMenu).toBeVisible();
  }

  async navigateTo(section) {
    await this.navMenu
      .getByRole('link', { name: section })
      .click();
  }
```

```
  async search(query) {
    await this.searchBox.fill(query);
    await this.searchBox.press('Enter');
  }

  async logout() {
    await this.logoutBtn.click();
    await this.page.waitForURL('/login');
  }

  async getCurrentUser() {
    return await this.userBadge.textContent();
  }
}
```

CheckoutPage — complex multi-step flow

```
// pages/CheckoutPage.js
export class CheckoutPage {

  constructor(page) {
    this.page = page;
    // Step 1: Cart
    this.checkoutBtn    = page.getByRole('link', { name: 'Checkout' });
    this.cartItems      = page.getByTestId('cart-item');
    this.cartTotal      = page.getByTestId('cart-total');
    // Step 2: Shipping info
    this.firstNameInput = page.getByLabel('First name');
    this.lastNameInput  = page.getByLabel('Last name');
    this.postalInput    = page.getByLabel('Postal code');
    this.continueBtn    = page.getByRole('button', { name: 'Continue' });
    // Step 3: Order summary
    this.finishBtn      = page.getByRole('button', { name: 'Finish' });
    this.orderConfirm   = page.getByText('Thank you for your order');
  }

  async proceedToCheckout() {
    await this.checkoutBtn.click();
    await expect(this.page).toHaveURL(/checkout-step-one/);
  }

  async fillShippingInfo(firstName, lastName, postalCode) {
    await this.firstNameInput.fill(firstName);
    await this.lastNameInput.fill(lastName);
    await this.postalInput.fill(postalCode);
    await this.continueBtn.click();
    await expect(this.page).toHaveURL(/checkout-step-two/);
  }

  async getOrderTotal() {
    const text = await this.cartTotal.textContent();
    return parseFloat(text.replace('$', ''));
  }
```

```
  async placeOrder() {
    await this.finishBtn.click();
    await expect(this.orderConfirm).toBeVisible();
  }

  async completeCheckout(firstName, lastName, postalCode) {
    await this.proceedToCheckout();
    await this.fillShippingInfo(firstName, lastName, postalCode);
    await this.placeOrder();
  }
}
```

Using POM pages in tests

```
// tests/checkout/purchase-flow.spec.js
import { test, expect } from '@playwright/test';
import { LoginPage }    from '../../pages/LoginPage';
import { DashboardPage} from '../../pages/DashboardPage';
import { CheckoutPage } from '../../pages/CheckoutPage';

test.describe('Purchase flow', () => {

  test('complete checkout with valid data', async ({ page }) => {
    const login    = new LoginPage(page);
    const dashboard = new DashboardPage(page);
    const checkout  = new CheckoutPage(page);

    // Login
    await login.goto();
    await login.loginSuccessfully('standard_user', 'secret_sauce');

    // Verify dashboard loaded
    await dashboard.isLoaded();

    // Navigate to a product and add it to cart
    await page.goto('/inventory.html');
    await page.locator('[data-test="add-to-cart-sauce-labs-
backpack"]').click();

    // Complete checkout
    await page.locator('[data-test="shopping-cart-link"]').click();
    await checkout.completeCheckout('Alice', 'Smith', '12345');

    console.log('Order placed successfully');
  });

  test('login with invalid credentials shows error', async ({ page }) => {
    const login = new LoginPage(page);
    await login.goto();
    await login.login('invalid@test.com', 'wrongpass');
    await expect(login.errorMessage).toBeVisible();
    const msg = await login.getErrorMessage();
    expect(msg).toContain('Username and password do not match');
  });
```

```
});
```

- **playwright.dev — Page Object Model —** https://playwright.dev/docs/pom

DAY 19 Data-Driven Testing — JSON, CSV, and Parameterised Tests

Data-driven testing runs the same test logic with multiple input sets. This is essential for login scenarios (valid user, invalid user, locked user), form validation (empty fields, invalid formats, edge cases), and any feature where behaviour changes based on data.

JSON test data files

```
// data/users.json
[
  {
    "description": "admin user — full access",
    "email": "admin@test.com",
    "password": "admin123",
    "role": "admin",
    "expectedPage": "/dashboard",
    "canDelete": true,
    "canEdit": true
  },
  {
    "description": "viewer user — read only",
    "email": "viewer@test.com",
    "password": "viewer123",
    "role": "viewer",
    "expectedPage": "/dashboard",
    "canDelete": false,
    "canEdit": false
  },
  {
    "description": "invalid credentials — should fail",
    "email": "nobody@test.com",
    "password": "wrongpass",
    "role": null,
    "expectedPage": "/login",
    "canDelete": false,
    "canEdit": false
  }
]
```

Parameterised tests with a for loop

```
import { test, expect } from '@playwright/test';
```

```
import users from '../data/users.json';
import { LoginPage } from '../pages/LoginPage';

// Create one test per user entry
for (const user of users) {
  test(`Login: ${user.description}`, async ({ page }) => {
    const login = new LoginPage(page);
    await login.goto();
    await login.login(user.email, user.password);

    if (user.role) {
      // Valid user — should reach dashboard
      await expect(page).toHaveURL(user.expectedPage);
      await expect(page.getByText(user.role)).toBeVisible();

      // Check permissions
      if (user.canDelete) {
        await expect(page.getByRole('button', { name: 'Delete'
})).toBeVisible();
      } else {
        await expect(page.getByRole('button', { name: 'Delete'
})).not.toBeVisible();
      }
    } else {
      // Invalid user — should stay on login
      await expect(page).toHaveURL('/login');
      const error = await login.getErrorMessage();
      expect(error).toBeTruthy();
    }
  });
}
```

Form validation with data-driven tests

```
// data/form-validations.json
[
  { "field": "email",     "value": "",             "error": "Email is
required" },
  { "field": "email",     "value": "notanemail",    "error": "Invalid email
format" },
  { "field": "password",  "value": "abc",          "error": "Password too
short" },
  { "field": "password",  "value": "12345678",     "error": null }
]

// In test file
import validations from '../data/form-validations.json';

for (const v of validations) {
  test(`Form validation: ${v.field} = '${v.value}'`, async ({ page }) => {
    await page.goto('/register');
    await page.getByLabel(v.field).fill(v.value);
    await page.getByRole('button', { name: 'Submit' }).click();

    if (v.error) {
```

```
      await expect(page.getByText(v.error)).toBeVisible();
    } else {
      await expect(page.getByText(v.error)).not.toBeVisible();
    }
  });
}
```

- **playwright.dev — Parameterise tests —** https://playwright.dev/docs/test-parameterize

DAY 20 API Testing — HTTP Requests, Auth, and Hybrid Tests

Playwright's request fixture lets you make HTTP API calls from within tests. This serves two purposes: (1) test your REST APIs directly, and (2) use API calls to create/clean up test data faster than UI interactions.

Basic API tests — GET, POST, PUT, DELETE

```
import { test, expect } from '@playwright/test';

// Base URL for API (can be different from UI base URL)
const API = 'https://reqres.in/api';

test.describe('Users API', () => {

  test('GET /users — returns paginated list', async ({ request }) => {
    const res = await request.get(`${API}/users?page=1`);

    // Status assertion
    expect(res.status()).toBe(200);
    expect(res.ok()).toBeTruthy(); // true if status 200-299

    // Parse and assert body
    const body = await res.json();
    expect(body).toHaveProperty('data');
    expect(body.data).toBeInstanceOf(Array);
    expect(body.data.length).toBeGreaterThan(0);

    // Assert specific fields on first item
    const first = body.data[0];
    expect(first).toHaveProperty('id');
    expect(first).toHaveProperty('email');
    expect(first.email).toMatch(/@/); // email format check
  });

  test('GET /users/1 — returns single user', async ({ request }) => {
    const res = await request.get(`${API}/users/1`);
    expect(res.status()).toBe(200);
    const { data } = await res.json();
    expect(data.id).toBe(1);
    expect(data.first_name).toBeTruthy();
```

```
  });

  test('GET /users/999 — returns 404', async ({ request }) => {
    const res = await request.get(`${API}/users/999`);
    expect(res.status()).toBe(404);
  });

  test('POST /users — creates a user', async ({ request }) => {
    const res = await request.post(`${API}/users`, {
      data: {
        name: 'Alice QA',
        job: 'Automation Engineer'
      }
    });
    expect(res.status()).toBe(201);
    const body = await res.json();
    expect(body.name).toBe('Alice QA');
    expect(body.id).toBeTruthy();      // ID was assigned by server
    expect(body.createdAt).toBeTruthy();
  });

  test('PUT /users/1 — updates user', async ({ request }) => {
    const res = await request.put(`${API}/users/1`, {
      data: { name: 'Alice Updated', job: 'Senior QA' }
    });
    expect(res.status()).toBe(200);
    const body = await res.json();
    expect(body.name).toBe('Alice Updated');
    expect(body.updatedAt).toBeTruthy();
  });

  test('DELETE /users/1 — deletes user', async ({ request }) => {
    const res = await request.delete(`${API}/users/1`);
    expect(res.status()).toBe(204); // No content
  });

});
```

API testing with authentication

```
test.describe('Authenticated API', () => {

  // Store token across tests in a describe block
  let authToken;

  test.beforeAll(async ({ request }) => {
    // Login via API and store token
    const res = await request.post(`${API}/login`, {
      data: { email: 'eve.holt@reqres.in', password: 'cityslicka' }
    });
    expect(res.status()).toBe(200);
    const body = await res.json();
    authToken = body.token;
    console.log('Auth token acquired');
  });
```

```
  test('authenticated request with Bearer token', async ({ request }) => {
    const res = await request.get(`${API}/users/me`, {
      headers: {
        Authorization: `Bearer ${authToken}`,
        Accept: 'application/json'
      }
    });
    expect(res.status()).toBe(200);
  });

});
```

Hybrid tests — API setup + UI verification

The most powerful pattern: use the API to create test data (fast), then verify it appears correctly in the UI (thorough). Use API to clean up after tests.

```
test('new product appears in admin UI', async ({ page, request }) => {

  // STEP 1: Create test data via API (instant, no UI clicks needed)
  const createRes = await request.post('/api/products', {
    data: {
      name: 'Test Laptop Pro',
      price: 999.99,
      category: 'electronics'
    },
    headers: { Authorization: `Bearer ${process.env.API_TOKEN}` }
  });
  expect(createRes.status()).toBe(201);
  const { id } = await createRes.json();
  console.log('Created product ID:', id);

  // STEP 2: Verify in UI
  await page.goto('/admin/products');
  await expect(page.getByText('Test Laptop Pro')).toBeVisible();
  await expect(page.getByText('$999.99')).toBeVisible();

  // STEP 3: Clean up via API
  await request.delete(`/api/products/${id}`, {
    headers: { Authorization: `Bearer ${process.env.API_TOKEN}` }
  });

  // STEP 4: Verify deleted from UI
  await page.reload();
  await expect(page.getByText('Test Laptop Pro')).not.toBeVisible();
});
```

Setting up API request context globally

```
// playwright.config.js
export default defineConfig({
  use: {
```

```
    // Base URL for page.goto()
    baseURL: 'https://www.saucedemo.com',
    // Base URL for request.get/post etc.
    extraHTTPHeaders: {
      'Accept': 'application/json',
    },
  },
});
```

- **playwright.dev — API testing guide —** https://playwright.dev/docs/api-testing
- **playwright.dev — APIRequestContext —** https://playwright.dev/docs/api/class-apirequestcontext
- **ReqRes — practice API —** https://reqres.in
- **JSONPlaceholder — mock API —** https://jsonplaceholder.typicode.com

DAY 21 — CI/CD with GitHub Actions, Reporting, and Environment Variables

The final day covers deploying your tests into a Continuous Integration pipeline. Every code change should automatically trigger your test suite. GitHub Actions is free for public repos and very straightforward to configure.

Complete GitHub Actions workflow

```
# .github/workflows/playwright.yml
name: Playwright End-to-End Tests

# Triggers: run on push or pull request to main/develop
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  # Also allow manual trigger from GitHub UI
  workflow_dispatch:

jobs:
  playwright-tests:
    runs-on: ubuntu-latest

    steps:
      # Step 1: Checkout the code
      - name: Checkout repository
        uses: actions/checkout@v4

      # Step 2: Set up Node.js
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
```

```
          node-version: '20'
          cache: 'npm'  # cache node_modules

      # Step 3: Install dependencies
      - name: Install dependencies
        run: npm ci  # use ci not install (respects package-lock.json)

      # Step 4: Install Playwright browsers + Linux system deps
      - name: Install Playwright browsers
        run: npx playwright install --with-deps

      # Step 5: Run tests
      - name: Run Playwright tests
        run: npx playwright test
        env:
          CI: true
          BASE_URL: ${{ secrets.BASE_URL }}
          API_TOKEN: ${{ secrets.API_TOKEN }}

      # Step 6: Upload test report (always — even on failure)
      - name: Upload HTML report
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 30

      # Step 7: Upload traces on failure
      - name: Upload traces on failure
        uses: actions/upload-artifact@v4
        if: failure()
        with:
          name: test-results
          path: test-results/
          retention-days: 7
```

Environment variables and secrets

Never hardcode credentials in test files. Use environment variables — in local development, load from a .env file; in CI, use GitHub Secrets.

```
# .env file (local development only — add to .gitignore!)
BASE_URL=https://staging.yourapp.com
API_TOKEN=your-api-token-here
ADMIN_EMAIL=admin@test.com
ADMIN_PASSWORD=secret123

// In your tests — read from environment variables
const baseURL     = process.env.BASE_URL || 'https://localhost:3000';
const apiToken    = process.env.API_TOKEN;
const adminEmail  = process.env.ADMIN_EMAIL || 'admin@test.com';
const adminPass   = process.env.ADMIN_PASSWORD;

// In playwright.config.js
```

```
export default defineConfig({
  use: {
    baseURL: process.env.BASE_URL || 'https://localhost:3000',
  }
});
```

SECURITY

Add .env to .gitignore

The .env file contains sensitive credentials. Add it to .gitignore immediately. In GitHub Actions, use encrypted Secrets (Settings > Secrets) — they are never visible in logs.

Load .env in tests locally

```
# Install dotenv
npm install --save-dev dotenv

// At the top of playwright.config.js
import 'dotenv/config'; // loads .env automatically

export default defineConfig({
  use: {
    baseURL: process.env.BASE_URL,
  }
});
```

Multiple reporters

```
// playwright.config.js
export default defineConfig({
  reporter: [
    ['html', { open: 'never' }],       // HTML report (don't auto-open in CI)
    ['list'],                            // Console output — good for CI logs
    ['json', { outputFile: 'results.json' }], // Machine-readable
    ['junit', { outputFile: 'results.xml' }], // For Jenkins / Azure DevOps
  ]
});

# CLI override
npx playwright test --reporter=line   # concise output
npx playwright test --reporter=dot    # minimal dots
```

Sharding — run tests in parallel across multiple CI machines

```
# Split tests into 4 shards across 4 machines
# Machine 1:
npx playwright test --shard=1/4
# Machine 2:
npx playwright test --shard=2/4
# Machine 3:
```

```
npx playwright test --shard=3/4
# Machine 4:
npx playwright test --shard=4/4

# In GitHub Actions — matrix strategy
strategy:
  matrix:
    shard: [1, 2, 3, 4]
steps:
  - run: npx playwright test --shard=${{ matrix.shard }}/4
```

- **playwright.dev — CI/CD introduction —** https://playwright.dev/docs/ci-intro
- **playwright.dev — GitHub Actions —** https://playwright.dev/docs/ci#github-actions
- **playwright.dev — Sharding —** https://playwright.dev/docs/test-sharding
- **playwright.dev — Reporter API —** https://playwright.dev/docs/test-reporters

SECTION 6 — MANDATORY CAPSTONE PROJECTS

Mandatory Capstone Projects

These three projects are not optional. They consolidate everything from all 21 days and represent the minimum work you need to produce to say you have learned Playwright at an intermediate level. Build them in order — each one adds complexity.

01 Google Search Automation

Site: https://www.google.com — this is a public site with no login required. Good for practicing navigation, dynamic elements, and assertions.

Requirements

13. Handle the cookie consent dialog if it appears (use try/catch with a 3 second timeout)
14. Perform a search using getByRole — NOT by CSS selector
15. Assert the URL contains the search query parameter
16. Assert at least 5 search result items are visible
17. Click the first organic result and assert the new page loads
18. Scroll down the results page and assert more results become visible
19. Take a full-page screenshot after the search results load
20. Write a second test that searches for a second term and validates the results are different

File structure

```
tests/
  google/
    search.spec.js
pages/
  GooglePage.js
```

02 SauceDemo E-Commerce Automation

Site: https://www.saucedemo.com — Credentials: standard_user / secret_sauce (also try locked_out_user and problem_user)

Requirements

21. Implement a LoginPage POM class with goto(), login(), and getError() methods
22. Implement an InventoryPage POM class with methods to add products by name
23. Implement a CartPage POM class with methods to get item count and proceed to checkout
24. Implement a CheckoutPage POM class covering all 3 checkout steps
25. Write a test for successful full checkout flow from login to order confirmation
26. Write a test for locked_out_user — assert the error message text
27. Write a data-driven test that adds multiple products from a JSON file
28. Write a test that verifies cart count updates correctly after adding each item
29. Add beforeEach to handle login, afterEach to take screenshot on failure

Test data file

```
// data/products.json
[
  { "name": "Sauce Labs Backpack",   "price": 29.99 },
  { "name": "Sauce Labs Bike Light",  "price": 9.99 },
  { "name": "Sauce Labs Bolt T-Shirt","price": 15.99 }
]
```

03 Full Automation Framework (Capstone)

This project must be built from scratch. It combines every skill from all 3 weeks into a professional-grade framework.

Framework requirements

30. Full POM layer: LoginPage, DashboardPage, ProductPage, CartPage, CheckoutPage
31. Custom fixture for authenticated sessions using test.extend()
32. Data-driven login tests using data/users.json with at least 3 user types
33. At least one API test that creates test data before a UI test runs
34. Hybrid API+UI test that verifies backend changes appear in the frontend
35. playwright.config.js with: baseURL, trace:'on-first-retry', multiple browsers, retries:2 in CI
36. GitHub Actions workflow file in .github/workflows/playwright.yml
37. HTML reporter configured with retention of at least 14 days
38. .env file with environment variables (not committed to git)
39. README.md explaining how to install, configure, and run the tests

Expected folder structure

```
framework/
  .github/
    workflows/
      playwright.yml
```

```
pages/
  BasePage.js         # optional base class for shared methods
  LoginPage.js
  DashboardPage.js
  ProductPage.js
  CartPage.js
  CheckoutPage.js
fixtures/
  auth-fixture.js
tests/
  auth/
    login.spec.js
    logout.spec.js
  shop/
    purchase-flow.spec.js
  api/
    products-api.spec.js
data/
  users.json
  products.json
.env               # NOT committed
.gitignore
playwright.config.js
package.json
README.md
```

SECTION 7 — CRITICAL RULES & ANTI-PATTERNS

Critical Rules & Anti-Patterns

These rules separate beginner Playwright code from professional Playwright code. Memorise them, enforce them in code review, and check for them whenever a test starts failing intermittently.

RULE 1

Master locators before anything else

Use getByRole and getByLabel as default locators. Only fall back to CSS selectors when no semantic locator exists. Never use XPath as your primary strategy — it is brittle and fails on minor DOM changes.

RULE 2

Never use waitForTimeout()

waitForTimeout() is a fixed sleep — it makes tests slow and fragile. If 3 seconds is not enough one day, the test fails. Instead: use expect().toBeVisible(), waitForURL(), waitForResponse(), or waitForSelector(). These retry until the condition is true.

RULE 3

Use the Trace Viewer before guessing

Every time a test fails in CI, download the trace.zip artifact and open it in trace.playwright.dev. The trace shows exactly what the browser saw, what actions were taken, and what network requests were made. Guessing wastes time.

RULE 4

Build projects during learning, not after

Apply each concept on the same day you learn it. Theory without immediate practice evaporates within 48 hours.

RULE 5

Codegen is for bootstrapping, not for finishing

Use npx playwright codegen to get initial selectors quickly. Then refactor: replace CSS with getByRole, add meaningful assertions, remove hardcoded waits.

RULE 6

Never commit API keys or passwords

Use environment variables and GitHub Secrets. Add .env to .gitignore. Rotate credentials immediately if they are ever committed by mistake.

Anti-patterns — code to avoid

Anti-pattern (avoid)	Better approach
`waitForTimeout(3000)`	`expect(el).toBeVisible()`
`page.click('#btn')`	`page.getByRole('button').click()`
`locator('//div[@id=...')`	`getByRole / getByLabel`
`Hardcoded credentials in test file`	`process.env.PASSWORD`
`No assertions — just actions`	`Assert after every meaningful action`
`test.only() committed to repo`	`Playwright's forbidOnly: !!CI`
`UI for test data setup`	`API calls in beforeAll`
`500+ line spec file`	`Split by feature into multiple files`

SECTION 8 — COMPLETE RESOURCE LIBRARY

Complete Resource Library

Every resource referenced in this module, organised by topic. All links verified at time of writing.

Official Playwright Documentation

- **Getting started — installation and first test —** https://playwright.dev/docs/intro
- **Writing tests — test syntax and structure —** https://playwright.dev/docs/writing-tests
- **Locators — complete locator guide —** https://playwright.dev/docs/locators
- **Other locators — XPath, CSS, id —** https://playwright.dev/docs/other-locators
- **Best practices —** https://playwright.dev/docs/best-practices
- **Assertions reference —** https://playwright.dev/docs/test-assertions
- **Auto-waiting and actionability —** https://playwright.dev/docs/actionability
- **Frames (iframes) —** https://playwright.dev/docs/frames
- **Pages (multiple tabs) —** https://playwright.dev/docs/pages
- **Network interception —** https://playwright.dev/docs/network
- **Dialogs —** https://playwright.dev/docs/dialogs
- **File upload and download —** https://playwright.dev/docs/input
- **Navigations and loading states —** https://playwright.dev/docs/navigations
- **Trace Viewer —** https://playwright.dev/docs/trace-viewer
- **Codegen — record and playback —** https://playwright.dev/docs/codegen
- **UI Mode —** https://playwright.dev/docs/test-ui-mode
- **Debugging guide —** https://playwright.dev/docs/debug
- **Fixtures —** https://playwright.dev/docs/test-fixtures
- **Page Object Model —** https://playwright.dev/docs/pom
- **Parameterize tests —** https://playwright.dev/docs/test-parameterize
- **API testing —** https://playwright.dev/docs/api-testing
- **Configuration reference —** https://playwright.dev/docs/test-configuration
- **CI/CD introduction —** https://playwright.dev/docs/ci-intro
- **GitHub Actions —** https://playwright.dev/docs/ci#github-actions
- **Sharding —** https://playwright.dev/docs/test-sharding
- **Reporters —** https://playwright.dev/docs/test-reporters
- **Test annotations (skip, tag, etc.) —** https://playwright.dev/docs/test-annotations
- **Command-line interface —** https://playwright.dev/docs/test-cli
- **Browsers — Chromium, Firefox, WebKit —** https://playwright.dev/docs/browsers
- **Mobile emulation —** https://playwright.dev/docs/emulation

JavaScript Fundamentals

- **MDN — Complete JavaScript Guide —** https://developer.mozilla.org/en-US/docs/Web/JavaScript
- **MDN — async function reference —** https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
- **MDN — Promises —** https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
- **MDN — Arrow functions —** https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions
- **MDN — Destructuring assignment —** https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment
- **javascript.info — Modern JavaScript Tutorial (complete) —** https://javascript.info
- **javascript.info — Promises, async/await —** https://javascript.info/async-await
- **javascript.info — Arrow functions —** https://javascript.info/arrow-functions-basics

Practice and Learning Sites

- **DemoQA — forms, widgets, drag-and-drop, web tables —** https://demoqa.com
- **The Internet Herokuapp — comprehensive UI patterns —** https://the-internet.herokuapp.com
- **SauceDemo — e-commerce practice site —** https://www.saucedemo.com
- **ReqRes — REST API for testing —** https://reqres.in
- **JSONPlaceholder — mock REST API —** https://jsonplaceholder.typicode.com
- **Online Playwright Trace Viewer —** https://trace.playwright.dev

Courses and Video Learning

- **Test Automation University — Playwright with JavaScript (free) —** https://testautomationu.applitools.com/playwright-intro/
- **Playwright Solutions — YouTube channel —** https://www.youtube.com/@playwrightsolutions
- **Playwright official YouTube demos —** https://www.youtube.com/results?search_query=playwright+testing

Tools and Development Environment

- **VS Code — download —** https://code.visualstudio.com
- **Playwright Test for VS Code extension —** https://marketplace.visualstudio.com/items?itemName=ms-playwright.playwright
- **Node.js — download (use LTS version) —** https://nodejs.org/en/download
- **nvm — Node Version Manager (macOS/Linux) —** https://github.com/nvm-sh/nvm
- **nvm-windows —** https://github.com/coreybutler/nvm-windows
- **Git — version control —** https://git-scm.com
- **GitHub — repository hosting —** https://github.com

GitHub and CI/CD References

- **GitHub Actions documentation —** https://docs.github.com/en/actions
- **GitHub Secrets — storing credentials —** https://docs.github.com/en/actions/security-guides/encrypted-secrets
- **Playwright on GitHub Actions — official example —** https://playwright.dev/docs/ci#github-actions

Source Code and Community

- **Playwright GitHub repository (source + issues) —** https://github.com/microsoft/playwright
- **Playwright Discord community —** https://discord.gg/playwright-807756831384403968
- **Playwright on Stack Overflow —** https://stackoverflow.com/questions/tagged/playwright

After completing this module you will be able to:

✓ Build a complete automation framework from scratch using industry best practices
✓ Write reliable, non-flaky tests using Playwright's semantic locator system
✓ Handle any real-world UI pattern: iFrames, file uploads, tabs, dialogs, custom dropdowns
✓ Write data-driven tests using JSON and parameterised test loops
✓ Test REST APIs directly and combine API and UI steps in a single test
✓ Debug failing tests confidently using the Trace Viewer and UI Mode
✓ Deploy tests to run automatically on every code push using GitHub Actions
✓ Apply the Page Object Model pattern for maintainable, scalable test code